**A3: Small Object Detection Using YOLO**

**Github Link: https://github.com/amacharla15/CSCI611_Akshith_Macharla**

## 1. Introduction

In this assignment, I implemented small object detection using YOLOv8n for traffic light detection. Small object detection is difficult because the target object occupies only a very small portion of the image, especially when it is far away from the vehicle-mounted camera. As a result, pretrained object detectors often miss the object or produce false positives. My goal in this project was to first evaluate a pretrained YOLOv8n model on traffic light images, then fine-tune the model on a traffic-light dataset and compare the results before and after training.

## 2. Dataset Description and Preprocessing

I used the **Bosch Small Traffic Lights Dataset (BSTLD)** as the dataset for this project. BSTLD is a public dataset for traffic light detection, and it is a good match for the assignment because traffic lights in the images are often small and distant.

The original BSTLD annotations were provided in YAML format rather than YOLO format. Because YOLO requires one text file per image in the format:

<class_id> <x_center> <y_center> <width> <height>

I wrote a preprocessing script to convert the original BSTLD labels into YOLO format. The original labels in BSTLD included many traffic light states such as Red, Yellow, Green, RedLeft, and similar categories. Since the goal of this assignment was small object detection rather than fine-grained traffic light state classification, I merged all of those states into a single class called traffic_light.

After preprocessing, I created a random split of the dataset into training, validation, and test sets:

- Train: **3565 images**, **7525 bounding boxes**
- Validation: **763 images**, **1588 bounding boxes**
- Test: **765 images**, **1643 bounding boxes**

This conversion and split process satisfied the requirement to prepare the dataset in YOLO format and to obtain test results on a held-out subset.

## 3. YOLO Environment Setup

I set up the YOLO environment using Python, Ultralytics YOLO, OpenCV, NumPy, and PyTorch. I verified that the environment was working correctly by loading the pretrained yolov8n.pt model

and running inference on a sample image. After confirming that inference worked correctly, I moved to dataset preparation and training.

**4. Baseline Evaluation Using Pretrained YOLOv8n**

Before training, I evaluated the pretrained yolov8n.pt model on my BSTLD test split. This baseline step was important because it gave me a “before fine-tuning” reference point.

Since the pretrained YOLOv8n model was trained on COCO classes, I filtered the predictions so that only the traffic-light detections were compared against my one-class ground-truth labels. I then computed true positives, false positives, and false negatives using IoU-based matching.

The pretrained baseline results were:

- True Positives (TP): **748**
- False Positives (FP): **736**
- False Negatives (FN): **895**
- Precision: **0.5040**
- Recall: **0.4553**
- F1-score: **0.4784**

These baseline results showed that the pretrained model was able to detect some traffic lights, but performance was limited. The model missed many true traffic lights and also produced many false detections. This was expected because the pretrained model was generic and had not been fine-tuned for the BSTLD traffic light domain.

**5. Fine-Tuning the YOLO Model**

I fine-tuned YOLOv8n using the one-class BSTLD dataset that I prepared. I trained the model with the following main setting:

- Model: yolov8n.pt
- Image size: **640**
- Epochs completed: **15**
- Batch size: **16**

I originally launched training for more epochs, but I stopped at 15 because the validation metrics had already improved significantly and later improvements were becoming smaller. The

goal of the assignment was to demonstrate fine-tuning and improvement, not to fully optimize the model for production deployment.

During training, the validation metrics improved steadily. At epoch 15, the validation performance reached approximately:

- mAP50: **0.7144**
- mAP50-95: **0.3655**

This showed that the model was learning to localize traffic lights much better after fine-tuning on the BSTLD dataset.

**6. Evaluation of the Fine-Tuned Model**

After training, I evaluated the saved best.pt model on the same BSTLD test split using the same TP/FP/FN evaluation method as the pretrained baseline. This ensured that the comparison was fair and consistent.

The fine-tuned model results were:

- True Positives (TP): **1125**
- False Positives (FP): **329**
- False Negatives (FN): **518**
- Precision: **0.7737**
- Recall: **0.6847**
- F1-score: **0.7265**

Compared to the pretrained baseline, the fine-tuned model improved by:

- Precision improvement: **+0.2697**
- Recall improvement: **+0.2295**
- F1 improvement: **+0.2481**

This was a strong improvement and clearly demonstrated that fine-tuning on a dataset of small traffic-light objects significantly improved detection quality.

**7. Confidence Threshold and NMS Optimization**

After obtaining the trained model, I further tested different confidence thresholds and NMS IoU values to optimize the final detection behavior. This step was important because inference settings can change the balance between precision and recall.

I tested:

- Confidence thresholds: **0.15, 0.25, 0.40**
- NMS IoU values: **0.45, 0.60, 0.70**

The best balanced result was obtained with:

- Confidence threshold = **0.25**
- NMS IoU = **0.45**
- Precision = **0.8309**
- Recall = **0.6817**
- F1-score = **0.7489**

I also observed that:

- Lower confidence thresholds produced more detections and slightly higher recall, but also increased false positives.
- Higher confidence thresholds improved precision by reducing false positives, but recall dropped because more true traffic lights were missed.
- A confidence threshold of **0.40** and NMS IoU of **0.45** produced the highest precision (**0.9017**), but it did not give the best balanced F1-score.

Therefore, I selected **confidence = 0.25** and **NMS IoU = 0.45** as the best final configuration for balanced performance.

**8. Qualitative Analysis: False Positives and False Negatives**

I saved prediction images from both the pretrained baseline and the fine-tuned model to visually inspect the results.

From the saved images, I observed the following:

**False negatives:**

- Very small or distant traffic lights were sometimes missed.
- Traffic lights with weak contrast against the background were harder to detect.

- Some lights partially occluded by poles or other scene elements were not detected.

**False positives:**

- Bright background objects or reflective regions could sometimes be mistaken for traffic lights.
- Distant light-like structures in the environment occasionally triggered detections in the pretrained model more often than in the fine-tuned model.

After fine-tuning, the model clearly became better at focusing on the actual traffic light regions, reducing false positives and improving localization quality.

**9. Challenges Faced**

The main challenges in this project were:

1. **Small object size**
   Traffic lights occupied very few pixels, making them hard to detect accurately.
2. **Dataset conversion**
   BSTLD was not already in YOLO format, so I had to convert the original YAML annotations into YOLO label files.
3. **Pretrained class mismatch**
   The pretrained COCO YOLO model uses its own class IDs, so I had to carefully evaluate only the traffic-light predictions against my one-class ground-truth labels.
4. **Threshold trade-offs**
   Improving precision often lowered recall, so I needed to experiment with confidence threshold and NMS settings to find a balanced configuration.

**10. Suggestions for Improving Small Object Detection**

There are several directions that could improve this project further:

1. **Train with higher image resolution**
   Using image size 1280 could help because small traffic lights would occupy more pixels.
2. **Run more training experiments**
   Additional experiments with longer training or different hyperparameters could further improve performance.
3. **Use stronger augmentation**
   More targeted augmentations such as scale changes, blur, and viewpoint shifts could help the model generalize better to small traffic lights.

4. **Extend to multiple classes**
   Instead of merging all traffic-light states into one class, future work could classify red, yellow, green, and directional arrows separately.

5. **Test on real driving video footage**
   The final detector could be evaluated on real street-driving video sequences, such as the Chico street footage mentioned in the assignment.

**11. Conclusion**

In this project, I implemented small object detection using YOLOv8n for traffic light detection. I used the Bosch Small Traffic Lights Dataset, converted its annotations into YOLO format, and created train/validation/test splits. I first evaluated the pretrained YOLOv8n model to establish a baseline, and then fine-tuned the model on the prepared traffic-light dataset.

The fine-tuned model substantially outperformed the pretrained baseline. The F1-score improved from **0.4784** to **0.7265**, and further tuning of confidence threshold and NMS improved the best balanced F1-score to **0.7489**. These results showed that dataset-specific fine-tuning and inference tuning are both important for improving small object detection performance.

Overall, this assignment demonstrated that YOLO can detect small traffic lights more effectively after fine-tuning on a domain-specific dataset and after carefully selecting inference parameters.